

THE KOSELLECK MACHINE

Testing the Sattelzeit as a Network Event

A method note on the Koselleck Machine: measuring whether English vocabulary reorganized as a system, and not only word by word, across 1770-1830.

REPOSITORY koselleck-networks

DOCUMENT docs/method.tex → docs/method.pdf

CORPUS SPAN 1510-1910, uniform 20-year windows

DISPLAY RESOLUTION per-variant, auto-picked (1,500-word cap); sweep: 15 settings

COMPILED September 8, 2026

Contents

1 Context	2
2 Method	2
2.1 Corpus	6
2.2 Not a closed dataset	6
3 The interactive explorer	7
3.1 How the labels are actually produced	8
3.1.1 Choosing the labeling resolution	8
3.1.2 The prompt	9
3.1.3 Generating and checking labels	12
3.1.4 Where the published labels live	13
4 Current limitations	13

1 Context

The historian Reinhart Koselleck argued that the period 1770-1830 (the **Sattelzeit**, German for “saddle period”) was not simply an era in which many words happened to change meaning at the same time. He claimed it was a moment when the whole vocabulary of politics and society reorganized **as a system**: concepts shifted together, in relation to one another, rather than drifting independently one at a time. This has always been argued in prose, from close reading of texts. It has never been tested computationally.

Ryan Heuser’s *Computing Koselleck* took the first step towards such a test: he trained a separate word-embedding model for each historical period and measured how far each individual word moves from one period to the next. He found that this movement is unusually large around 1770-1830, which supports the idea that something real happened in that window. But his method looks at one word at a time. It cannot say whether words moved **together**, as a system reorganizing around a shared structure, or whether each word’s shift is its own independent event that simply happens to cluster in time.

This project adds that second, missing test. Instead of asking how far a single word moves, it asks whether the **groupings** among words, the way the whole vocabulary is organized into clusters of related meaning, change unusually around the same decades. If Koselleck is right, we should see the cluster structure itself reorganize at the Sattelzeit, not just individual words drifting off on their own.

2 Method

The method reuses the same per-period training that Heuser’s approach uses, so the two studies remain directly comparable, and adds a network layer on top that a word-by-word approach cannot produce.

1. **Split the corpus into time windows.** The corpus is divided into even 20-year windows, currently 1510 to 1910, starting at 1510 rather than 1500 so that both 1770 and 1830 land exactly on a period boundary - the transitions that open and close the Sattelzeit are each a literal edge between two periods, not an approximation. The windows are uniform across the whole timeline, not just around the Sattelzeit: measuring more finely right where reorganization was expected and more coarsely everywhere else would bias the method toward finding exactly what it was looking for. Heuser’s own results show a second spike of word-level change around 1905-1925, unrelated to the Sattelzeit, a pattern a Sattelzeit-only sampling scheme could never

have picked up. The span reflects the corpus assembled for this project so far; the same pipeline applies unchanged to any other span or window size, set entirely in configuration, given a period-dated corpus to feed it.

2. **Train a language model per period.** For each time window, a model (**word2vec**) is trained independently, learning a numeric representation of every word from how it is actually used in that period's text: words that appear in similar contexts end up with similar representations. Training a separate model per period, rather than one model updated continuously, is what makes it possible to later measure both individual word drift and whole-system reorganization: a word's meaning only makes sense relative to the other words trained alongside it, in that same period.
3. **Build a word-similarity network.** From those representations, a network is built for each period: every word is a node, linked to its 15 closest neighbours by **cosine similarity** (a standard measure of how alike two words' representations are). A fixed number of nearest neighbours is used instead of a similarity threshold, because at this vocabulary scale a fixed threshold produces networks too dense to cluster meaningfully: at a 0.3 threshold, roughly one in ten possible word pairs was linked, tens to hundreds of millions of edges per period. Purely grammatical words (articles, prepositions, pronouns) and single-letter tokens are excluded as network nodes, since the interest is in words that carry conceptual content, not the grammatical scaffolding of the language (those grammatical words are still used during training itself, since they help place every other word correctly).
4. **Detect communities.** A **community detection** algorithm (**Leiden**) is run on each network, grouping words that are densely connected to one another. In practice, these groups function as semantic fields: sets of words that, in that period, are used in similar ways to each other more than to the rest of the vocabulary.

This differs from familiar clustering methods such as *k*-means in a basic way. *k*-means fixes a number of groups in advance and assigns each item to whichever group's average position it sits closest to. Leiden does neither: it does not fix the number of groups beforehand, and it never compares a word to a group's "average", it only ever asks a relational question about a word's actual connections. For any word, and any group it is already directly connected to, Leiden checks whether moving that word into the group would make the network as a whole denser *within* groups and sparser *between* them than plain chance would predict, given how connected that word and that group already are overall. If the move helps, it is made; if not, the word stays put or a different move is tried. Repeating this, word by word, until no single move can improve things any further, is the entire algorithm.

The number this process is trying to improve at every step has a name, **modularity**

(written Q). It compares how connected the words placed inside the same group actually are against how connected they would be expected to be if the same set of connections had been handed out at random instead. A grouping that captures real structure scores well above zero; a grouping no better than chance scores around zero; a grouping that forces genuinely unrelated words together scores *below* zero, actively worse than not grouping at all. Leiden's search is nothing more than hill-climbing on this one number.

5. **Repeat detection at several levels of detail.** Community detection does not have one single "correct" answer: a parameter called **resolution** controls how many groups appear and how large they are, by changing how large a word's own connections have to be, relative to chance, before joining a group is judged worthwhile. For that reason every network is processed at fifteen fixed resolution settings, 0.1 to 16.0, and any claim about reorganization has to hold up across all of them, not just one chosen after the fact.

Hard rule. No single setting among the fifteen is ever treated as "the" result and the rest discarded. The sweep's entire purpose is to check that the same conclusion holds broadly across settings, not to pick whichever one happens to agree with the hypothesis. No finding is reported if it only holds at a single, convenient resolution. A second, separate resolution decides only which single partition is displayed and labeled in the interactive explorer below - a reading convenience, with no bearing on which finding gets reported. It is picked per period and per region automatically, from the same swept values, as the lowest one at which no community grows past a 1,500-word cap; §3 covers the mechanism and why it replaced an earlier fixed choice.

6. **Compare consecutive periods.** For each pair of consecutive periods, the comparison is restricted to words that appear in both (each period has its own vocabulary). The best possible match between one period's groups and the next period's groups is found first, because group numbers do not mean the same thing from one period to the next, so the correspondence has to be established before anything can be compared. Concretely (`metrics.py`'s `align_communities`): build a contingency table counting, for every (old group, new group) pair, how many shared words fall in that pair, then solve it as a single global assignment problem (the Hungarian algorithm, via SciPy's `linear_sum_assignment`) that picks the one-to-one relabeling of new group numbers onto old ones maximizing the total word count matched. This is a fixed, deterministic optimum over all groups at once, computed once per period pair - there is no percentage-overlap threshold anywhere in it, and no per-group decision made independently of the others. A single shared word then counts as **moved** if, under that one relabeling, its new group's mapped-back old identity differs from the old group it was actually in; otherwise it counts as **stayed**. Then the fraction of

shared words that still ended up in a different group, despite that best match, is measured. That fraction, how much of the shared vocabulary “migrated” to a new group, is what this project calls the **migration fraction**: its central measurement of how much reorganization happened between two periods. The interactive explorer’s per-word “moved”/“stayed” arrows (Section 3) are read off this exact same per-word classification, not a separate or looser check, so a single word’s arrow and the aggregate migration fraction can never disagree about what counts as moving.

7. **Test whether reorganization concentrates at the Sattelzeit.** The hypothesis predicts that the migration fraction should be unusually high specifically for the transitions that fall inside or close out the 1770-1830 window, compared to every other transition across the full 1510-1910 timeline. This measurement has been run against the current corpus; results are reported separately (docs/pipeline_manual.pdf’s Stage 8 section) and are not summarized here.
8. **Repeat the whole measurement per region.** The corpus is not evenly British across the timeline, and the imbalance is worst exactly where it matters most: in 1770-1790 the American material (Evans-TCP, 1,030 documents) outweighs the British (mostly ECCO-TCP, 792 documents), because ECCO’s public transcriptions thin out sharply at the end of the eighteenth century. A change in the cluster structure across that boundary could therefore reflect a change in *who was writing* rather than a change in the language. To separate the two, every step above (training, network construction, community detection, the resolution sweep, and the migration fraction) is also run on each region’s documents alone, in addition to the combined corpus. If the reorganization is real, it should show up inside the British-only and the American-only timelines as well, not only in the mixture of them; if it shows up only in the combined run, it is an artifact of corpus composition rather than semantic history. Nothing here hardcodes which regions exist: a document’s region is simply the top-level folder its source was filed under, so the same check extends to any other split of the corpus.
9. **Dictionary cross-check (not yet implemented).** As a second line of evidence, independent of the network entirely, the plan is to take the words that change group right at the pivot and check whether the OED or other period dictionaries independently record a dated sense change at the same moment. This layer is not yet designed: the corpus mixes British and American sources, and matching each source against the wrong regional dictionary would invalidate the comparison, so the choice of dictionary per source has to be resolved first.

2.1 Corpus

The primary source is the **Text Creation Partnership (TCP)**: curated, manually corrected transcriptions with no OCR noise, covering 1500-1800 (EEBO-TCP, ECCO-TCP, Evans-TCP). Since the Sattelzeit extends to 1830 and TCP stops at 1800, the 1800-1900 gap is filled by the **British Library**'s digitised 19th-century books collection (49,455 books, roughly 65,000 volumes, OCR-derived text with catalogue metadata already attached, public domain). Project Gutenberg was considered first and dropped: its metadata carries only its own digitisation date, not the original print publication date, which makes it unusable for a method that buckets every document by the year it was actually written. HistWords (pre-trained historical embeddings) is used only as an independent sanity check, not as primary evidence.

Each source is filed under a region: `british` for EEBO-TCP, ECCO-TCP, and now the British Library supplement; `american` for Evans-TCP, which stops at 1800 along with the rest of TCP. The region tag is not a claim about two different English-language communities - for most of this timeline, American print culture and British print culture share the same language, and treating them as linguistically distinct would not be defensible. It tracks archive provenance instead: EEBO/ECCO and the British Library are one continuous British-print lineage, Evans is an independently curated American-print archive. Read this way, the region split is a robustness check, not a dialect comparison: if the reorganization signal this project measures shows up independently in both archives, that is evidence it is not an artifact of one archive's quirks; the combined corpus stays the headline result either way. A direct consequence of this design: since the British Library supplement is tagged `british`, the `american` region has no data past 1800 at all, an honest asymmetry rather than a gap to paper over.

2.2 Not a closed dataset

This project is a method. The corpus is one input to it, fetched separately and deliberately not vendored: a reader who disagrees with it, or has a better one, can rerun the whole measurement on their own text rather than take these numbers on trust. Swapping in a different corpus is either a folder drop (already-valid TCP P4 XML needs no code change) or one new reader function beside `iter_xml_members` in `src/parse_tcp.py`, emitting the same `{region, source, doc_id, year, text}` record the TCP path already writes - everything downstream reads only that normalised record and needs no change. README's Adding a new corpus section gives the full worked contract, including the exact JSON shape and the two concrete cases. There is no upload interface, hosted service, or API-key-driven ingestion: clone the repository, put corpus files in the right folder, rerun the pipeline scripts in order.

3 The interactive explorer

Alongside the pipeline, a small web tool, the **Koselleck Machine**, makes the per-period networks directly explorable, without reading raw numbers. It has three views of the same underlying data.

Timeline. The primary view. Type a word and see, in one continuous strip, which community it belonged to in every period it appears in: the community's plain-English label and its lane (below) as a secondary colour cue, an explicit gap where the word is simply absent from that period's vocabulary, and a marked step wherever the word's community actually changed from the period before, versus stayed the same. Earlier drafts of this tool led with the graph explorer below, but historian collaborators who reviewed it found a force-directed diagram hard to read at a glance; what they consistently asked for instead was exactly this, which domain a word belonged to, over time, read directly rather than inferred from a picture. Clicking any period in the strip opens that word's full neighbourhood there, reusing the word search view below.

Word search. The same underlying data, presented as a plain table instead of a diagram: type a word and a period, and get back its closest neighbours by similarity, each neighbour's community, and whether the searched word's own community changed since the previous period. Useful when the actual numbers matter more than the picture, and reused directly by the timeline's own per-period drill-in above.

Graph explorer. A period slider drives a force-directed network diagram: nodes are words, their size reflects how connected they are, and their color marks which community they belong to in that period. A search box focuses the view on one word and its neighbourhood; moving the slider one period at a time shows that neighbourhood evolving, and a change in a word's color between periods marks a change of community membership. A side panel explains the method in plain terms and shows, for the currently selected resolution, how the migration measurement holds up across all fifteen resolution settings - the same sweep described above. Kept for browsing the raw network structure, and because building it first is what surfaced this whole labeling and lane apparatus, but no longer the primary way to read the tool, per the reception above.

Region toggle. Where the per-region runs described above have been built, both views also offer a region switch (combined, British only, American only) so the same word in the same period can be compared across the sub-corpora directly, and a shift can be checked against the possibility that it only exists in the mixture. The switch lists only the regions that were actually built, read off the data itself rather than assumed.

Every community shown in any view carries a short, plain-English label (for example "Government & Law", "Moral Character & Virtue"), read by a language model from each

community's most representative words: a convenience for a human reader, not a finding in itself. A label is not re-read independently every period. Whenever the same correspondence step the migration fraction is built on maps a community back to a predecessor with a label, that label is a candidate to carry forward - but only after two independent model reads, given the inherited label and the community's current words, both agree it still fits; either reader saying no forces a fresh read instead. Across the three current snapshots (§3's "Where the published labels live"), 58–63% of labels in the combined and British runs carry forward this way; in the thinner American run, where more periods are a lineage's own first appearance, fresh reads are the majority instead. Roughly half of communities in the combined and British runs, and a third in American, are sorted into the **"Structural / Uncertain"** lane rather than a topical one: the corpus is multilingual (English, Latin, Law French, Welsh, Scots, and more) and early modern, and a community detected by this method sometimes groups words by shared grammar (verb conjugations, comparative forms, spelling variants) rather than by shared topic. Every view surfaces this lane directly next to the label.

Each community is additionally sorted into one **lane**: a small, fixed list of fifteen broad domains (government and law, religion, medicine, trade, and so on), plus one explicit "structural / uncertain" lane for exactly the "mixed" case above. The lane list itself was authored once by hand, from a read-through of every non-mixed label already produced, then fixed. A language model assigns each community to one of these existing lanes; it never invents a new one per run. That fixed-in-advance property is what lets a lane stay a meaningful, comparable signal across independent pipeline runs even though, as the last section explains, the underlying community structure and the raw label text do not reproduce identically run to run. In the tools above, a community's plain-English label and number stay the primary signal; its lane is a secondary colour cue layered on top, not a replacement for reading the actual label.

3.1 How the labels are actually produced

The paragraphs above describe what the labels are for. This subsection gives the mechanics, so that a technical reader can see exactly what the model was asked to do and reproduce or replace it. Labeling is an optional, clearly separable stage: every quantitative result in this document is produced by `src/metrics.py` and needs no labels at all.

3.1.1 Choosing the labeling resolution

Community detection produces a partition at every resolution in the sweep; the webapp displays and labels only one of them, a choice separate from the sweep itself and revisited once already. It was originally 1.0. Adding the British Library supplement grew

some periods' vocabularies past 170,000 words, and at 1.0 the largest community in the latest periods had grown with them to 24,608 words (14% of that period's vocabulary) - a size at which no label, model or human, can describe what is actually inside it. This was not a new failure mode introduced by the supplement, only a more visible one: the largest community was already a stable 12-15% of the vocabulary at every period size tested, small and unremarkable at 70,000 words, absurd at 170,000.

Modularity falls off smoothly as resolution rises, with no sharp elbow in the tested range, so there is no purely statistical answer for the right resolution - the real ceiling is practical, how many communities per period stay browsable, not mathematical. The choice was settled by a concrete before/after check rather than a formula. Take the community containing the word "system" in 1870-1890, the single worst offender: at resolution 1.0 its top words read as an unstructured mix (*perjury, arrogance, indigestion, sensuality, bronchitis, extortions, dyspepsia*); at 2.0 and 3.0 the same words are still there, effectively unchanged; only at **4.0** does the community actually split, and the half that keeps "system" becomes a real, nameable theme (*arrable, assessments, registration, payment, loans, forfeitures, coroners, judiciary, treasurers, sheriffs, freehold, manors, copyhold*) - civil administration and property law. The same check on two other periods at 4.0 (one small, one mid-sized) produced equally coherent results (philosophy/rhetoric/mathematics; church governance and clergy), so the resolution was raised project-wide rather than only for the largest periods. `leiden.resolution_sweep`, the range every quantitative claim is actually tested across, was extended to match (up to 4.0, from 2.0), so the display choice and the tested range stay aligned. Growing the corpus further should be expected to call for raising this again - the right resolution scales with vocabulary size, it is not a constant.

This manual check was later turned into an automatic rule, picked separately for every period and region-split variant against the same criterion (does the worst-case community actually stay readable, now a 1,500-word cap) rather than once for the whole corpus. `docs/pipeline_manual.pdf`'s Stage 6 section covers the search mechanism and the one edge case it took a full production run to surface.

3.1.2 The prompt

The prompt lives in the repository as a single Markdown file, currently `src/prompts/community_labeling_v3.md`, which is parsed at runtime by `src/label_communities.py` rather than duplicated as a string constant in code, so the file stays the one place the instructions and the lane list are edited. It has three sections: the fixed lane list, the system prompt, and the user message template. One model call is made per Leiden community, at the auto-picked display resolution described above.

The system prompt, quoted from that file, tells the model:

“You are labeling word clusters from a computational study of an early modern through nineteenth-century English-language corpus: the Text Creation Partnership (EEBO, ECCO, Evans; 1500-1800, clean manually-keyed transcription) plus the British Library’s digitised 19th-century books collection (1800-1900, OCR text). Each cluster is a ‘community’ found by running the Leiden algorithm on a per-period word-similarity network, words that are used in similar contexts more than they are used with the rest of that period’s vocabulary. The corpus is multilingual (English, Latin, Law French, Welsh, Scots, and other languages appear untranslated) and pre-modern, so a community sometimes groups words by shared grammatical form (verb conjugations, comparatives, spelling variants), by shared foreign language, or by proper names (people, places, biblical figures) rather than by a real topic. In periods drawn from the British Library supplement, a community can also be a cluster of OCR fragments - words with a missing prefix or suffix from a misread hyphen or scan artifact - rather than real words at all. *All of these are honest and expected outcomes when they are genuinely what the community is, not a failure to avoid.*”

It then asks the model to work through, in order, before deciding anything: what the community’s most central (*Core*) words suggest on their own; whether its less-central (*Mid-rank* and *Peripheral*) words support or undercut that reading, naming at least one specific word from each tier; and only then, whether a genuine topic holds across all three, or whether the community is better described as grammatical, foreign-language, or name-driven. That ordering is itself the point of a required reasoning field, added 2026-09-02 (see below). Once that reasoning is done, it asks for exactly three decisions:

1. “Whether most of the words shown - across every tier given, not just the Core - share a genuine topical or thematic connection a historian would recognize (e.g. law, medicine, religion, trade), as opposed to being grouped mainly by grammar, a shared non-English language, or being mostly proper names or fragments.”
2. “A short plain-English label, two to five words, for what the community is actually about. If the grouping is grammatical, foreign-language, or name-driven rather than topical, the label should say what kind of cluster it is (e.g. ‘Second-Person Verb Forms’, ‘Latin Text’, ‘Biblical Names’), not force a topic onto it.”
3. “Exactly one lane from the fixed list above. Use ‘Structural / Uncertain’ whenever the answer to (1) is no, do not stretch a grammatical, foreign-language, or name cluster into a thematic lane just because a few of its words are suggestive of one.”

The required reasoning field is a direct fix for a specific failure mode, not a general polish pass. An earlier version of this prompt asked for the same three decisions with no forced reasoning step, and too often reached for an easy grammatical or “Structural / Uncertain” read from the Core tier alone, without evidence the Mid-rank and Peripheral words had been weighed at all. Ordering reasoning before `label` and `lane` in the tool schema the model must call forces that weighing to happen, the same way a person

asked to show their work is less likely to skip a step than one asked only for a final answer. A blind check on two real communities, each sent to a fresh, context-free model twice, once with each version of the prompt, found the earlier version defaulting to “Structural / Uncertain” both times despite its own draft label already naming a real theme, and the current version reaching a concrete, evidence-grounded topical lane both times instead.

The user message carries only the material the model is meant to judge, filled in from the extraction step: the region, the period, the community’s size in words, and a sample of its member words by network-degree rank. That sample used to be a flat list of the 25 words with the highest degree in the community, its most central members, until a real gap surfaced: a flat top-slice only ever shows words tightly bound to a community’s core, hiding a community whose core looks coherent but whose lower-degree tail has drifted to a different topic entirely. The sample is now stratified instead: up to 15 words each from the top (Core), middle (Mid-rank), and bottom (Peripheral) of the ranked membership list, 45 words in total for any community large enough to split three ways; a smaller community is shown in full. The reasoning field described above exists specifically so a model has to look at all three tiers, not just the Core one this older flat sample used to show alone.

Two properties of this setup matter more than the wording. First, the lane vocabulary is closed. The model is not asked to invent a taxonomy; it must choose one of fifteen lanes fixed in advance, listed in the same file and reproduced here in the order the prompt gives them:

1. Government, Law & Administration
2. Religion, Theology & the Church
3. Morality: Virtue & Vice
4. Medicine, Body & Health
5. Science, Mathematics & Natural Philosophy
6. Nature, Landscape & Weather
7. Military & Warfare
8. Trade, Finance & Commerce
9. History, Genealogy, Nobility & Kinship
10. Rhetoric & Persuasion
11. Literature, Drama & Poetic Diction
12. Domestic Life, Dress & Household

- 13. Geography & Territory
- 14. Books, Learning & Scholarship
- 15. Structural / Uncertain

That list was not invented for the prompt either: it was derived by hand from 707 labels already produced across the combined, British, and American runs, then frozen. Second, the response is not free text. The model must answer by calling a tool, `assign_label`, whose schema takes a `label` string (“two to five word plain-English label”) and a `lane` constrained by a JSON-schema `enum` to exactly the fifteen lanes parsed out of the prompt file. A returned lane outside that list is rejected and the call retried, so a label with an out-of-taxonomy lane can never reach the data files.

3.1.3 Generating and checking labels

`extract_community_words.py` writes, for each populated period at the configured display resolution, every community’s size and its stratified Core/Mid-rank/Peripheral word sample (see above) to `communities/community_words_display[_<region>].json`, once for the combined corpus and once per region, since a region’s own Leiden run has its own community numbering. `label_communities.py generate` turns that JSON into a human-editable CSV, walking periods in chronological order; `docs/pipeline_manual.pdf`’s Stage 7 section gives the full command sequence and CSV schema, and README’s Labeling communities section has the same commands for a bare clone.

A community whose best correspondence to the period before it (the same correspondence step the migration fraction is built on) has a label is a *candidate* to inherit it, not an automatic inheritance: two independent model reads, given that inherited label and the community’s current words, are asked the same yes-or-no question, does this label still fit. Only unanimous agreement carries the label forward unchanged; either reader saying no forces a fresh read, exactly as a genesis community (no predecessor at all) gets one. This replaced an earlier, blunter safeguard, a fixed cap on how many periods a label could inherit before a forced re-read, itself built after a real case: a community genuinely holding Dutch and German text at 1510-1530 was still labeled “Dutch and German Text” fifteen periods later, by which point its content had passed through seven unrelated themes - including, at the end, English church governance - all unread. Checking the actual words on every candidate inheritance, rather than counting periods, catches that drift directly instead of bounding how long it can go unnoticed.

`compile` turns the CSV, including any hand edits, into `communities/community_labels_display[_<region>].json`, which is the exact file `webapp/app.py` reads. It also stamps a `_meta` block into that JSON recording the resolution, the region, the number of com-

munities, how many were human-edited, how many were inherited, and a standing caveat that the labels are a single model read-through and not an empirically validated taxonomy. Compiling used to append a literal “(mixed)” to the label text of every `Structural / Uncertain` community; that is no longer done. The `lane` field already carries the “this is not a real topic” signal on its own, and stacking the word “mixed” onto an already-decisive reading (“Second-Person Verb Forms”, “Biblical Book Abbreviations”) made even a clear description look like the model could not settle on one.

`publish` copies the CSV and the compiled JSON for a region out of the data directory and into this repository. `--region` also accepts a region name or `all` at every subcommand.

3.1.4 Where the published labels live

The label files that the pipeline writes live in the data directory, which is gitignored and does not travel with the repository. That is why a collaborator cloning the repo previously saw no community names at all. The `publish` subcommand exists to fix exactly that: it copies a snapshot into the repository’s `labels/` directory, which currently holds one CSV and one compiled JSON per region. The filenames no longer embed a resolution number: since the display resolution is now picked per variant, not once globally (“Choosing the labeling resolution” above), a single number in the filename would be meaningless once different periods in the same run can land on different resolutions.

```
labels/community_labels_display.csv
labels/community_labels_display.json
labels/community_labels_display_british.csv
labels/community_labels_display_british.json
labels/community_labels_display_american.csv
labels/community_labels_display_american.json
```

That is the citable snapshot: the version of the labels this document describes, readable without running the pipeline or holding the corpus. `publish` only copies files to disk; it never commits, so the snapshot is reviewed by hand before it is added to version control.

4 Current limitations

Corpus coverage. 1800-1900 is now covered by the British Library supplement described above, but not evenly: ECCO is thin after 1780, so 1770-1790 leans on more American than British documents (Evans-TCP, 1,030, against mostly-ECCO’s 792); the British Library supplement brings the next window, 1790-1810, close to parity instead. The `american` region has no data at all past 1800, since Evans-TCP stops there and

the supplement is British-only (see Corpus above). Per-region results carry wider uncertainty than the combined headline numbers, and the american region specifically cannot speak to anything past the Sattelzeit's closing edge.

OCR artifacts. The British Library text is OCR-derived, unlike TCP's manually corrected transcription. `ocr_refinement.py` repairs two classes of damage: an original line-break hyphen left sitting in the text ("*utrum- que*" for *utrumque*); and the harder case of the OCR dropping that hyphen entirely and leaving a bare space instead ("*par ticulars*" for *particulars*), fixed by merging two adjacent words only when the merged form is real and at least one half isn't independently real on its own - checked by hand against the one period a reference wordlist covers, 1790-1810 (142 correct merges in the first 120 changed documents). That wordlist is built only from that period's own TCP text, so periods entirely after 1800 remain unrepaired and untested by this fix. A different, unrelated source of noise - French and Welsh text surfacing as its own network community - is fixed separately, by dropping a document once a language classifier is at least 90% confident it isn't English.

Word-level noise in the network. Two further patterns pass through into the trained networks and cluster tightly enough by co-occurrence to form their own communities, neither yet fixed at the stage that would need to change (which word becomes a node). Latin citation-apparatus abbreviations (*capvt*, *deute*, *regu*) are correctly transcribed originals, not damage of any kind, but carry no topical content either; docs/pipeline_manual.pdf's Stage 5 section has the detail. Separately, raising the labeling resolution surfaced a distinct, unconfirmed TCP transcription pattern in a handful of communities from 1510-1650: a missing "con-"/"com-" prefix on otherwise-recognizable words (*mytted* for *committed*, *uersation* for *conversation*, *monwealth* for *commonwealth*), most likely a scribal abbreviation mark mishandled during text extraction - not confirmed against the raw XML, and no fix attempted.

Label reliability. A label is a candidate to inherit from its predecessor only after two independent model reads agree it still fits the community's current words (§3 above); either reader saying no forces a fresh read. Across the current label snapshots, that leaves 37-42% of labels in the combined and British runs, and the majority in American, as an independent read for a given pipeline run. That read is still a single, unvalidated pass by a language model, not a checked ground truth - useful for orientation, not for citation.

Reproducibility. Rerunning the full pipeline from scratch on the same corpus does not reproduce an identical result. Community detection itself is properly seeded and deterministic given a fixed network; the gap sits one step earlier, in training. Word2Vec training here uses multiple parallel threads for speed, and that kind of parallel training is not bit-for-bit repeatable even with a fixed random seed, because the exact

order in which threads update shared values depends on how the operating system happens to schedule them that run. Small differences in the trained word vectors cascade into a similarity network with slightly different edges, which cascades into a structurally similar but not identical community partition - in one earlier comparison, two full reruns of the same combined corpus produced community counts a few apart. This does not bear on the claim under test: reorganization is measured by whether `migration_fraction` concentrates at the Sattelzeit, checked across all fifteen resolutions within a single run, never by whether a specific community keeps its id or exact member words across two independently retrained copies of the pipeline. The one thing meant to stay comparable across independent reruns is the fixed fifteen-lane list, precisely because it is authored once by hand and never reinvented per run.